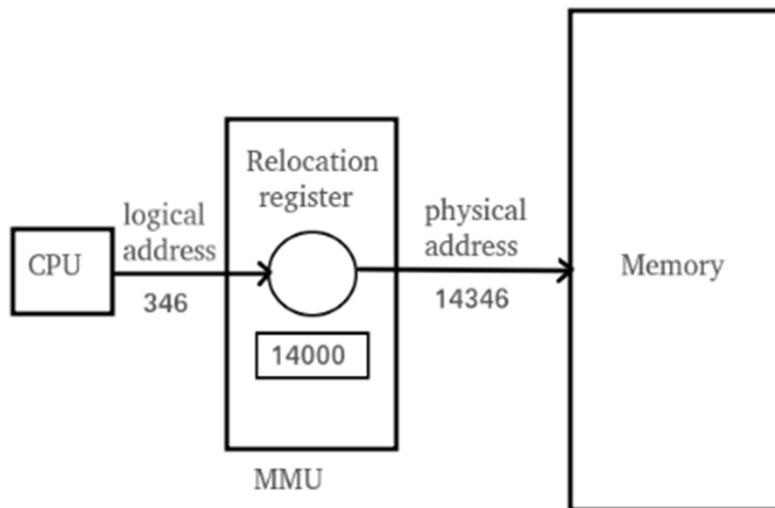


Unit: 4 Memory Management (Part 2)

Logical vs. Physical Address Space

- The address generated by CPU while running a program is known as Logical address.
- The logical address is virtual address, it doesn't exist physically, along these lines, that is why called as Virtual Address.
- This address is utilized as a kind of perspective to access the physical memory area by CPU.
- The term Logical Address Space is utilized for the arrangement of every logical address produced by a program's point of view.
- The hardware device called Memory-Management Unit is utilized for mapping logical address to its comparing physical address.
- Physical Address is similar to a physical area of required data in a memory.
- The user never legitimately manages the physical address however can access by its comparing logical address.
- The user program creates the logical address and imagines that the program is running in this logical address however the program needs physical memory for its execution, in this way, the logical address must be mapped to the physical address by MMU before they are utilized.
- The term Physical Address Space is utilized for every single physical addresses relating to the logical addresses in a Logical address space.

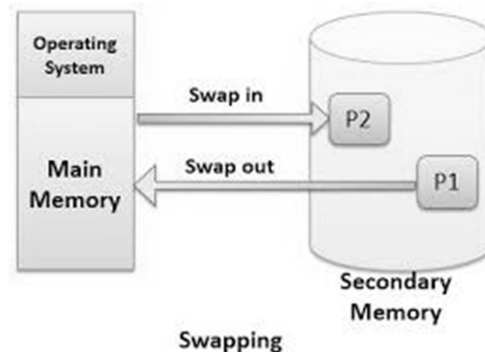


Mapping virtual-address to physical-addresses

Swapping

- Swapping is a technique wherein a process can be swapped incidentally out of primary memory (or move) to secondary storage (disk) and make that memory accessible to different processes.

- At some later time, the system swaps back the process from the secondary storage to primary memory.
- Swapping is a valuable method that empowers a computer to execute programs and control data files bigger than main memory.
- The operating system copies however much data as could reasonably be expected into main memory, and leaves the rest on the disk.
- At the point when the operating system needs data from the disk, it trades a bit of data (called a page or section) in main memory with a bit of data on the disk.
- In spite of the fact that performance is typically influenced by swapping process yet it helps in running numerous and huge processes in parallel and that is the reason Swapping is otherwise called a technique for memory compaction.
- Swapping includes performing two operations called swapping in and swapping out.
- The operation of setting the pages or blocks of data from the hard disk to the main memory is called swapping in.
- Then again, the operation of evacuating pages or blocks of data from main memory to the hard disk is known as swapping out.
- The swapping method is helpful when bigger program is to be executed or a few operations need to performed on a huge file.



- For swapping the sweeper additionally utilizes some address those are otherwise called the logical and physical address of processes. For giving the logical address into the physical address there the accompanying two methodologies utilized.
 1. **Static movement:** In the static migration swapping never to be performed in light of the fact that the process are consistently have a memory. This isn't to be changed at the execution time. For instance the memory which is given to the input and output tasks and the memory which is given to the CPU to executing the processes won't be changed with the goal that this is otherwise called the static migration of the memory.
 2. **Dynamic migration:** The dynamic migration is otherwise called the memory which is changed and moves by the processes at the time of execution. So that there must be some system to give memory those processes those are running. The mapping of the logical address into the physical address will be performed at the time of execution or at run time.

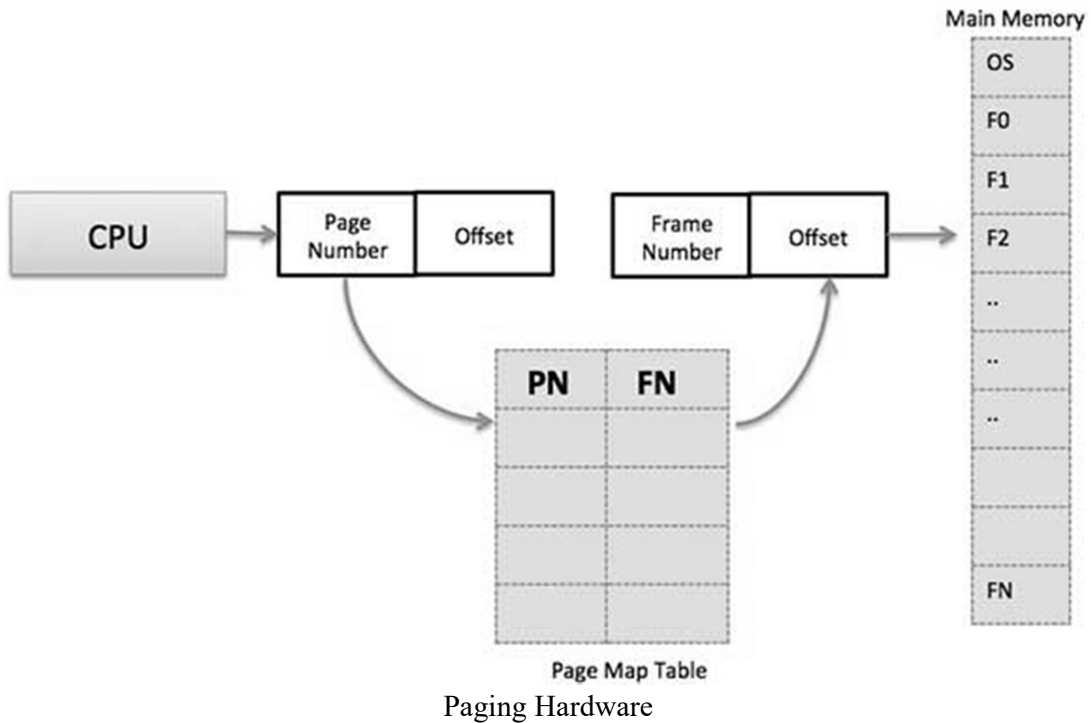
- At the point when the memory is given to the process then the specific location is put away by the CPU which is additionally have a section into the segment portrayal table methods where segments a process is running since when the process needs swapping that specific location must be reloaded.

Paging

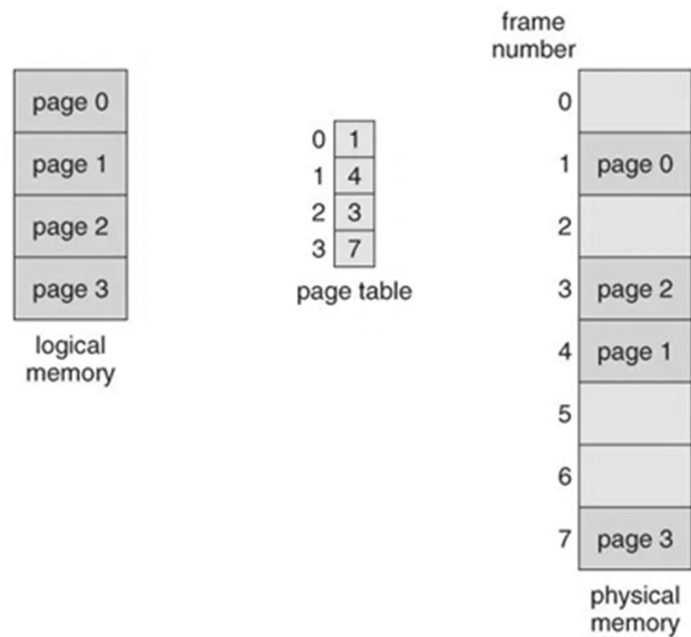
- In Operating Systems, Paging is a storage component used to recover processes from the secondary storage into the main memory as pages.
- The primary thought behind the concept of paging is to isolate each process as pages. The main memory will likewise be partitioned as frames.
- One page of the process is to be fitted in one of the frames of the memory. The pages are supposed to be store at the various locations of the memory however the need is consistently to locate the contiguous frames or holes.
- In this design, the operating system recovers data from secondary storage in same-size blocks called pages. Paging is a significant piece of virtual memory usage in current operating systems, utilizing secondary storage to give programs a chance to surpass the size of accessible physical memory.
- Support for paging has been taken care of by hardware. Nonetheless, ongoing structures have actualized paging by intently coordinating the hardware what's more, operating system, and particularly on 64-bit microprocessors.

Basic Method

- The fundamental technique for actualizing paging includes breaking physical memory into fixed-sized blocks called frames and breaking logical memory into blocks of a similar size known as pages.
- When a process needs to be executed, its pages are stacked into any accessible memory frames from their source (a file system or the support store).
- The backing store is partitioned into fixed-sized blocks that are of the similar size as the memory frames.
- The hardware support for paging is represented in figure below. Each address created the CPU is partitioned into two sections: a page number (p) and a page offset (d).



- The page number is utilized as a list into a page table. The page table contains the base location of each page in physical memory.
- This base location is joined with the page offset to characterize the physical memory address that is sent to the memory unit. The paging model of memory is appeared in figure below.



- The page size (like the frame size) is characterized by the hardware. The size of a page is regularly a power of 2, differing between 512 bytes and 16 MB for every page, contingent upon the computer architecture.
- The choice of a power of 2 as a page size makes the interpretation of a logical address into a page number and a page offset especially simple.
- In the event that the size of the logical address space is 2^m , and a page size is 2^n addressing units (bytes or words) then the high-request $m - n$ bits of a logical address assign the page number, and the n low-request bits assign the page offset.
- Along these lines, the logical address is as per the following:

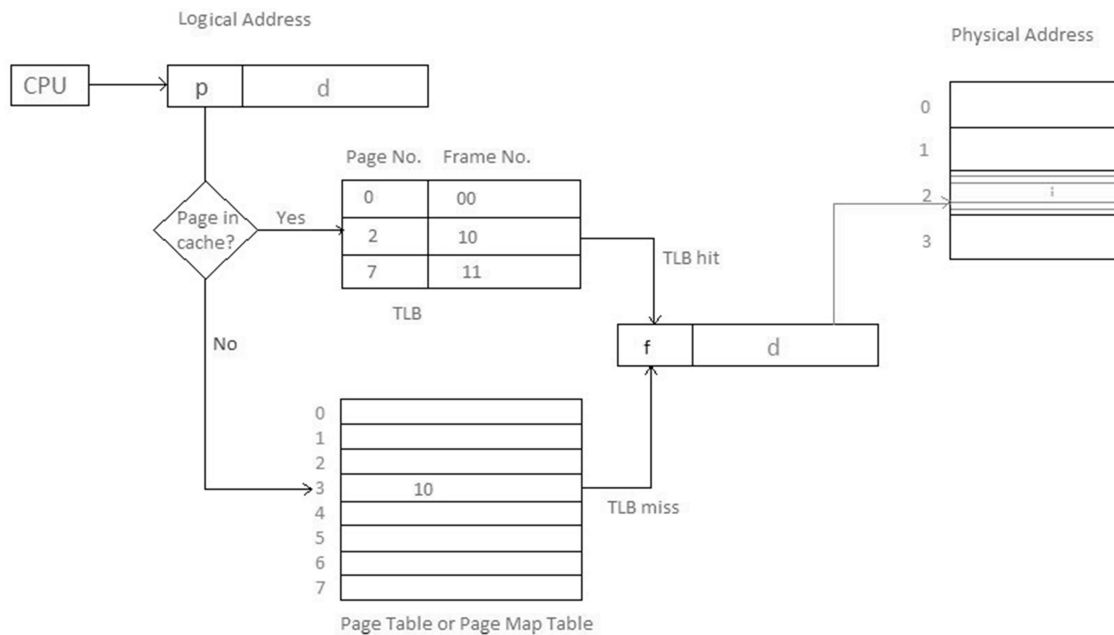
page number	page offset
p	d
$m - n$	n

where p is a index into the page table and d is the displacement inside the page.

- Since the operating system is overseeing physical memory, it must know of the assignment subtleties of physical memory-which frames are allotted, which frames are accessible, what number of total frames there are, etc.
- This data is commonly kept in a data structure considered a frame table. The frame table has one section for each physical page frame, demonstrating whether the last mentioned is free or designated and, on the off chance that it is allotted, to which page of which process or processes.

Hardware Support

- The hardware usage of page table should be possible by utilizing committed registers. Be that as it may, the use of register for the page table is agreeable just if page table is small.
- On the off chance that page table contain enormous number of sections, at that point we can utilize TLB (translation Look-aside cushion), an extraordinary, little, quick look into hardware cache.
- The TLB is acquainted, fast memory.
- Every passage in TLB comprises of two sections: a tag and a value.
- At the point when this memory is utilized, a item is compared with all tags at the same time. If the item is discovered, at that point comparing value is returned.



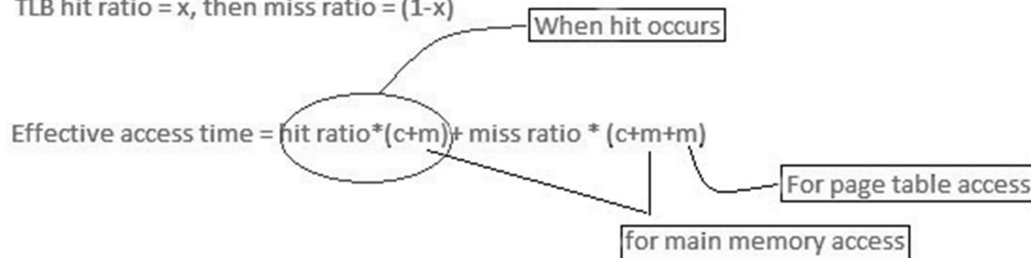
Main memory access time = m

In the event that page table are kept in main memory,

Successful Access time = m(for page table) + n(for specific page in page table)

TLB access time = c

TLB hit ratio = x, then miss ratio = (1-x)

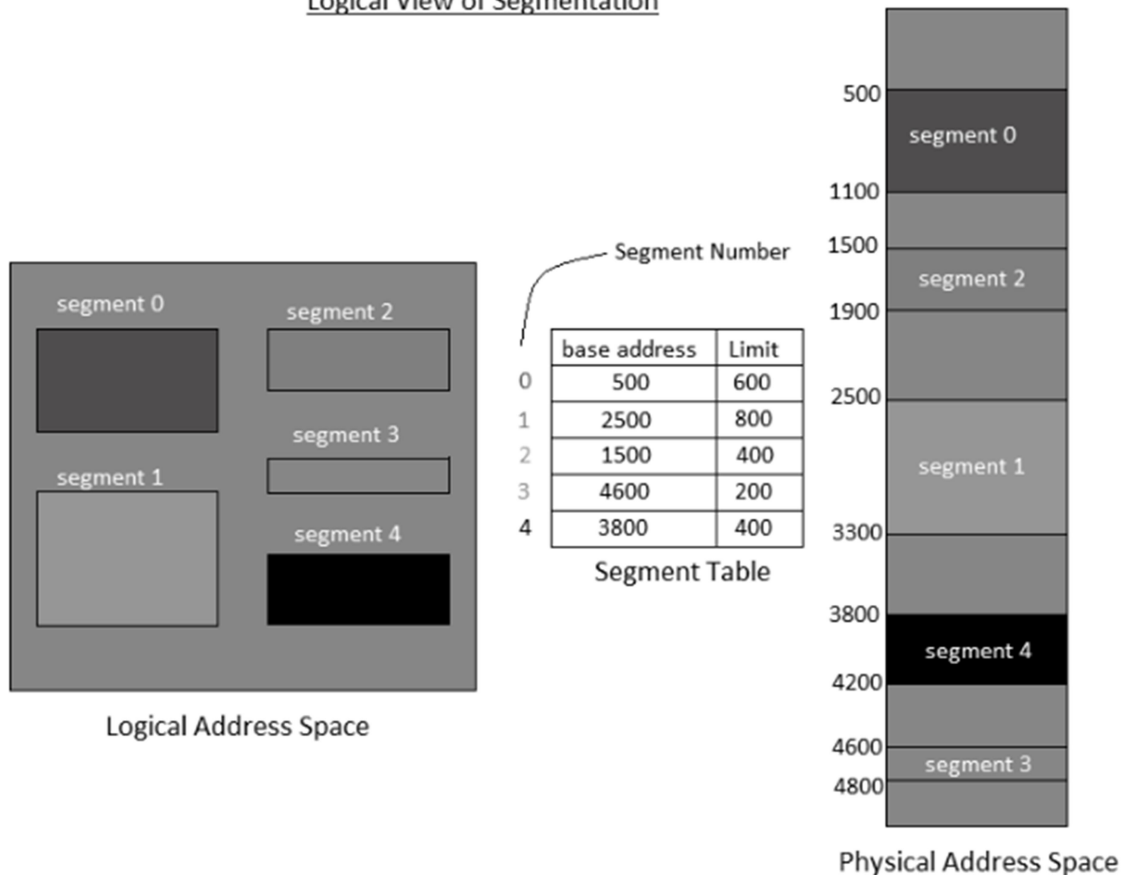


Segmentation

- Memory segmentation is a computer (primary) memory management method of division of a PC's primary memory into segments or sections.
- In a computer system utilizing segmentation, a reference to a memory area incorporates a value that distinguishes a segment and an offset (memory area) inside that segment.
- Segments or sections are likewise utilized in object files of compiled programs when they are connected together into a program image and when the image is stacked into memory.
- A process is isolated into Segments. The chunks that a program is partitioned into which are not really the majority of similar sizes are called segments.
- Segmentation gives user's perspective on the process which paging does not give.

- Here the user's view is mapped to physical memory.
- There are kinds of segmentation:
 1. Virtual memory segmentation – Each process is partitioned into various segments, not which are all inhabitant at any one point in time.
 2. Simple Segmentation – Each process is partitioned into various segments, which are all stacked into memory at run time, however not really contiguously.
- There is no straightforward connection between logical addresses and physical addresses in segmentation. A table is maintained to store the data about all such segments and is called Segment Table.
- Segment Table – It maps two-dimensional Logical address into one-dimensional Physical address. Its each table entry has:
 - Base Address: It contains the beginning physical address where the fragments reside in memory.
 - Limit: It determines the length of the fragment.

Logical View of Segmentation



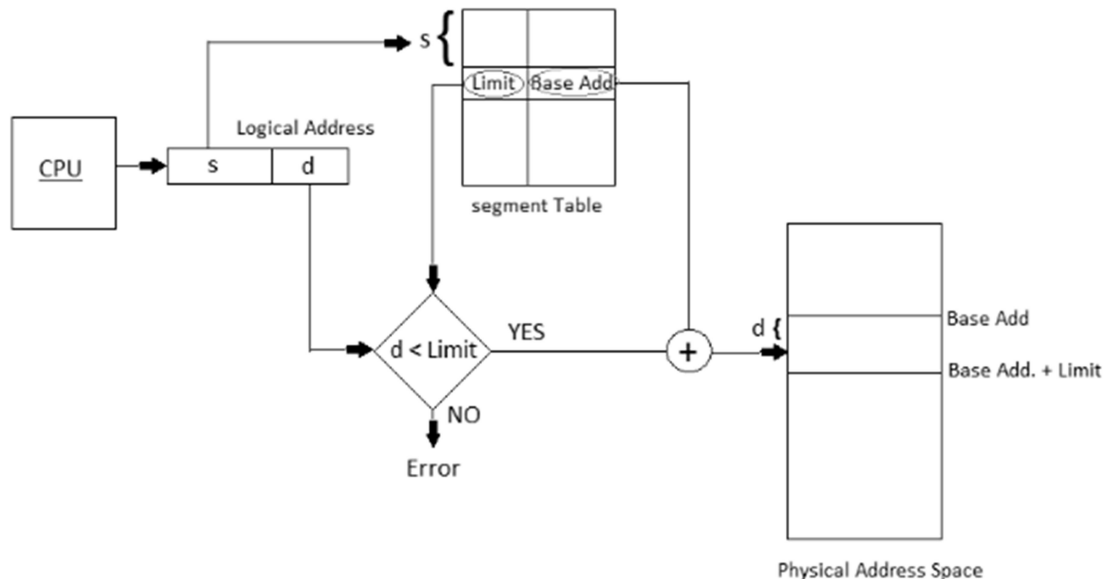
- Segments as a rule compare to regular divisions of a program, for example, singular schedules or data tables so division is commonly more noticeable to the programmer than paging alone.

- Different segments might be made for various program modules, or for various classes of memory utilization, for example, code and data portions. Certain segments might be shared between programs.

Why Segmentation is required?

- Till now, we were utilizing Paging as our main memory management system. Paging is more near to operating system instead of the User.
- It isolates all the process into the type of pages regardless to the way that a process can have some overall parts of functions which should be stacked in a similar page.
- Operating system couldn't care less about the User's perspective on the process. It might separate a similar function into various pages and those pages could conceivably be stacked simultaneously into the memory. It diminishes the productivity of the system.
- It is smarter to have segmentation which partitions the process into the segments. Each segment contain same sort of capacities, for example, main function can be incorporated into one segment and the library capacities can be incorporated into the other segment.

Translation of Logical Address into Physical Address



- Address generated by the CPU is partitioned into:
 - Segment number (s): Number of bits required to speak to the portion.
 - Segment balance (d): Number of bits required to speak to the size of the fragment.

Advantages-

- No Internal fragmentation.

- Segment Table consume less space in contrast with Page table in paging.

Disadvantages-

- As processes are stacked and expelled from the memory, the free memory space is broken into little pieces, causing External fragmentation.